


Explore | Expand | Enrich

# <Codemithra />™

<Codemithra />™

ETHNUS
Explore | Expand | Enrich

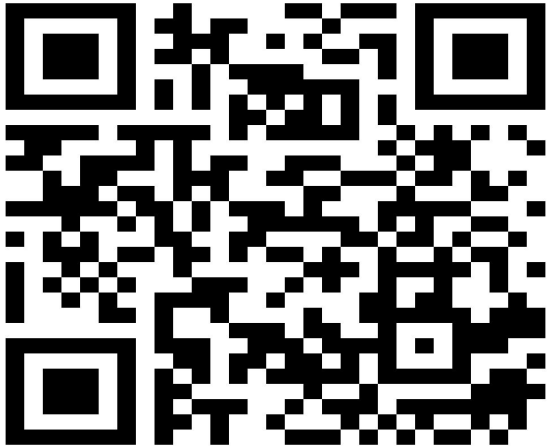
# LEXICOGRAPHICALLY FIRST PALINDROMIC STRING



# TEST TIME ON MAJORITY ELEMENT

**URL:**https://forms.gle/SFDVg26roZ2rtzcy5

**QR CODE:**



# Introduction

Imagine you have been given a string of alphabets.

The task is to rearrange the characters of the given string to form a lexicographically first palindromic string

## Lexicographically First / Lexicographically Minimum

String x is lexicographically less than string y

If either x is a prefix of y (and $x \neq y$) Or,

There exists such i ($1 \leq i \leq \min(|x|, |y|)$),

that $x_i < y_i$, and for any j ($1 \leq j < i$) $x_j = y_j$

The lexicographic comparison of strings is implemented by operator < in modern programming languages.

Our output string must be lexicographically minimum or lexicographically first.

**Example :**

Sample Input : str = “aabcc”

Sample Output : str = “acbca”

Print -1, if there are no such lexicographically minimum substring.

## Simple Approach

- Sort the string characters in alphabetical(ascending) order.
- One by one find lexicographically next permutation of the given string.
- The first permutation which is palindrome is the answer.

## Algorithm

1. Store frequency of each character in the given string

2. Check whether a palindromic string can be formed or not using the properties of palindromic string mentioned above.

3. If palindromic string cannot be formed, return “No Palindromic String”.

4. Else we create three strings and then return;

**front_str + odd_str + rear_str**

```
class EthCode {
 static char MAX_CHAR = 26;
 // Function to count frequency of each char in the
 // string. freq[0] for 'a',...., freq[25] for 'z'
 static void countFreq(String str, int freq[], int len)
 {
 for (int i = 0; i < len; i++)
 {
 freq[str.charAt(i) - 'a']++;
 }
 }
 // Cases to check whether a palindr0mic
 // string can be formed or not
 static boolean canMakePalindrome(int freq[], int len)
 {
 // count_odd to count no of
 // chars with odd frequency
 int count_odd = 0;
 for (int i = 0; i < MAX_CHAR; i++)
 {
 if (freq[i] % 2 != 0)
 {
 count_odd++;
 }
 }
 // For even length string
 // no odd freq character
 if (len % 2 == 0)
 {
 if (count_odd > 0)
 { return false;
 }
 else
 { return true;
 } }
```

```
// For odd length string
// one odd freq character
if (count_odd != 1)
{
return false;
}
return true;
}
// Function to find odd freq char and
// reducing its freq by 1returns "" if odd freq// char is
not present
static String findOddAndRemoveItsFreq(int freq[])
{
String odd_str = "";
for (int i = 0; i < MAX_CHAR; i++)
{
if (freq[i] % 2 != 0)
{
```

```
freq[i]--;
odd_str = odd_str + (char) (i + 'a');
return odd_str;
}}
return odd_str;
}
// To find lexicographically first palindromic
// string.
static String findPalindromicString(String str)
{
int len = str.length();
int freq[] = new int[MAX_CHAR];
countFreq(str, freq, len);
if (!canMakePalindrome(freq, len))
{
return "No Palindromic String";
}
```

```
// Assigning odd freq character if present
// else empty string.
String odd_str = findOddAndRemoveItsFreq(freq);
String front_str = "", rear_str = " ";
// Traverse characters in increasing order
for (int i = 0; i < MAX_CHAR; i++)
{
String temp = "";
if (freq[i] != 0)
{
char ch = (char) (i + 'a');
// Divide all occurrences into two
// halves. Note that odd character
// is removed by findOddAndRemoveItsFreq()
for (int j = 1; j <= freq[i] / 2; j++)
{
 temp = temp + ch;
}
```

```
// creating front string
front_str = front_str + temp;
// creating rear string
rear_str = temp + rear_str;
}
}
// Final palindromic string which is
// lexicographically first
return (front_str + odd_str + rear_str);
}

public static void main(String[] args)
{
String str = "malayalam";
System.out.println(findPalindromicString(str));
}}
```

Time Complexity : O(n)

Auxiliary Space Complexity : Constant

# Interview questions

1.Explain the problem statement of the "Lexicographically First Palindromic String" problem.

The problem involves finding the smallest palindromic string that can be formed using the characters of a given string `s`, and the resulting string should be lexicographically smallest among all possible palindromic strings.

# Interview questions

2.How does the proposed solution work?

The solution sorts the characters of the input string, and then iterates through the sorted characters. It appends each character to both the beginning and the end of the resulting palindromic string. This way, it constructs a palindromic string in a way that maintains lexicographical order.

# Interview questions

3. Can you explain the time complexity of the solution?

The time complexity is dominated by the sorting step, which takes O(n * log n) time, where n is the length of the input string. The subsequent iteration through the sorted characters takes O(n) time. So, the overall time complexity is O(n * log n).

# Interview questions

4. What happens if the input string contains duplicate characters?

Duplicate characters are handled correctly in the solution. The sorting step ensures that duplicate characters are grouped together, and the iteration through the sorted characters appends each character twice, maintaining their duplicates in the resulting palindromic string.



https://learn.codemithra.com

<Codemithra />™

eTHNUS™
Explore | Expand | Enrich

# THANK YOU

+91 78150 95095 | codemithra@ethnus.com | www.codemithra.com